Data Structures 1 – Winter 2025

Assignment A4 (Final)

General Rules

Item	Details
Deadline	**January 13, 2026, 11:59 p.m.**
Late Penalty	**No late submissions accepted.** (End of term)
Group Size	Pairs (two students)
Oral Defense	You may be asked to explain your solution *orally*. Failure to answer satisfactorily will result in a grade of 0 for each exercise discussed. Dividing tasks is **not** an excuse. You **must** be able to explain every exercise.
Academic Integrity	Any detected plagiarism or copying will be reported to the UG office. Copy/Paste from IA is plagiarism.
Language & Environment	Java (must compile in VS Code and pass all provided tests).
Deliverables	One **zip** file containing all Java source files (*wet part*).One **PDF** file with all written answers (*dry part*). We won't accept hand written assignments. The dry part must be typewritten in computer.

---

1. Connecting Cities – 20 pts (Dry/Wet)

A government plans to connect 8 cities (A-H) with a high-speed rail network. The potential tracks and their construction costs (in millions) are listed below.

Edges: (A,B, 10), (A,C, 5), (B,C, 12), (B,D, 4), (C,D, 8), (C,E, 3), (D,E, 15), (D,F, 6), (E,F, 8), (E,G, 7), (F,G, 9), (F,H, 11), (G,H, 2).

a. **(4 pts)** Compute the minimum network that connects all cities (hint: use Kruskal's)

b. **(8 pts)** Due to a strategic decision the government **mandates** that the expensive track between **(B, C)** (Cost 12) *must* be built and included in the network.

- Describe the algorithm/modification needed to satisfy this constraint while keeping the total cost as low as possible.
- What is the new set of edges and the new total cost?

Hint: Think about how the inclusion of this edge affects the cycle property for other edges.

c. **(3 pts)** Is the solution found in part (a) unique? If yes, explain why. If no, list one alternative edge that could replace an existing edge in the solution without changing the total cost.

d. **(5 pts)** Implement solutions for a. and b. in Java. Provide at least one meaningful test.

---

2. DevOps Pipeline (Topological Sort) – 15 pts (Dry/Wet)

Consider the following software build tasks and their dependencies:

- **1:** Linting (No deps)
- **2:** Unit Tests (Requires 1)
- **3:** Integration Tests (Requires 2)
- **4:** Security Scan (Requires 1)
- **5:** Build Docker Image (Requires 2 and 4)
- **6:** Deploy to Staging (Requires 3 and 5)

- **7:** E2E Tests (Requires 6)
- **8:** Promote to Prod (Requires 7)

a. **(3 pts)** Model the dependencies using a DAG (Directed Acyclic Graph).

b. **(3 pts)** Provide solutions of **Topological Sort** using **DFS** and using **BFS**

c. **(5 pts)** Assuming that non-dependent tasks can run in parallel, what is the minimum number of steps required to complete the pipeline? (Assume each task takes 1 time step). List the tasks running (including those in parallel) at each step.

d. **(4 pts)** Implement the items b. and c. in Java. Provide at least one meaningful test.

---

3. The "Rendezvous Point" – 25 pts (Wet)

Scenario: Two emergency response teams are located at different nodes in a city graph: Team Alpha at node `n1` and Team Bravo at node `n2`. They need to meet in a point `R`, a **Rendezvous Point** to consolidate equipment before a mission.

Objective: Find the node `R` such that the time it takes for **both** teams is minimized.

a. **(7 pts)** Provide the pseudocode of an algorithm of a function `findRendezvousPoint(n1,n2)` that solves the problem (i.e., return the node `R`).

b. **(8 pts)** Show the algorithm is correct and provide its complexity. If your algorithm relies partially of algorithms given in the lecture, show can assume that those algorithms works correctly.

c. **(10 pts)** Implement the algorithm in Java. Provide meaningful tests.

Hint: Think how Dijkstra could help.

---

4. Blockchain Analytics – 40 pts (Wet)

Instead of modifying the core Blockchain system, you will now build an Analytics Tool that interprets the blockchain history as a graph to detect suspicious patterns.

Problem Description We suspect that some users are engaging in "Circular Trading" (Money Laundering). This happens when money moves in a cycle (e.g., A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A) to artificially inflate transaction volume or hide the source of funds and there are some people that is accumulating coins from other people.

You will work with a `BlockchainAnalytics` that will help you to detect *transaction cycles* and *money mules*, accounts that receives funds from many different sources and may use for illegal actions.

4.1 Class `BlockchainAnalytics` The class will include a graph with all transactions of the blockchain history (excluding reverted transactions). The graph will help you identify flow between addresses.

Each node in the graph represent an addressed and the edges transactions with amounts. For instance a transaction:

- from: "123", to: "456", amount = 100

will create the nodes "123" and "456" and an edge ("123", 100, "456").

- **Constructor:** `BlockchainAnalytics(UBlockChain bc)`.
  - Expected complexity O(T)

4.2 Pattern Detection Implement the following methods in `BlockchainAnalytics`:

- `detectCycle()`:
  - Detects if there is a money laundering loop (e.g., A B C A).
  - Return the list of addresses involved in the cycle (e.g., `["Alice", "Bob", "Charlie", "Alice"]`).

  - It considers only *simple* cycles. That is, cycles when only two nodes are repeated once and should not include any nodes outside the cycle (e.g., `["Alice", "Bob", "Alice", "Bob", "Alice"]` and `["Messi", "Alice", "Bob", "Charlie", "Alice"]` are not valid)
  - Expected complexity: O(V+E)

- `detectShortestCycle()`:
  - Similar to the previous one but in case of existing more than one cycle, select the one with the *least* cost.
  - The cost is calculated by the sum of amounts of the transactions appearing in the loop.
  - Justify the complexity of your best solution

- `String findMoneyMule()`:
  - A *Money Mule* is defined here as an account that receives funds from many sources but sends funds to very few.
  - Return the address with the *highest* ratio: `received funds/sent funds`. Notices that if the address has not sent any funds, the ratio is directly the amount of `received funds`.
  - Note that the initial balance of address "0" should not be considered as received funds.
  - Expected complexity: O(V+E)

Notes:

- You may need to include functionality to ask the blockchain for all addresses and transactions.
- You may need to either adapt the graph to support addresses as vertices or use some mechanism in the `BlockchainAnalytics` to translate addresses to vertices.
- You may need to remove or add edges from the graph during your computation (e.g, cycles), but remember to leave the graph in the original state.

Implementation Details

- Use your own structures. You cannot use Java's built-in classes.
- You must implement the code using the given code template.
- You can use and extend the class `GraphAL` (Adjacency List) structure provided in the tutorial to support the analytics engine for your `UBlockChain`. It already supports graphs with weights.

- Add new test of the solutions.
- The solution need to pass the provided tests.
- Use of IA, and other forms of plagiarism are strictly forbidden.
